

TX REVERT 원인과 안전한 복구 설계

REVERT · TIMEOUT · REORG · Reason Classification · Recovery Policy

M3 TX 상태머신

S18

Day 05

COINCRAFT
ACADEMY

Session 18 아젠다

강의 20분 + 실습 35분

§ 1 · 3종 장애 이해

- REVERT vs TIMEOUT vs REORG
- 발생 시점 및 원인 비교
- 재시도 가능 여부 판단

§ 2 · REVERT 처리

- handleFailed 흐름
- reason 분류 정책
- 잘못된 상태 전이 방어

§ 3 · 실습

- 3종 장애 시뮬레이션
- handleTxRevert 구현
- FAILED 중복 안전성

학습 목표

- › [1] 3종 장애(REVERT·TIMEOUT·REORG) 구분 및 특성 이해
- › [2] handleTxRevert 구현 및 FAILED 전이 로직 작성
- › [3] reason 문자열 분류(BALANCE·PAUSED·ACCESS·UNKNOWN)
- › [4] 잘못된 상태 전이 시 InvalidStatusTransitionError 발생 확인
- › [5] FAILED 중복 호출에 대한 명시적 에러 처리 이해

01

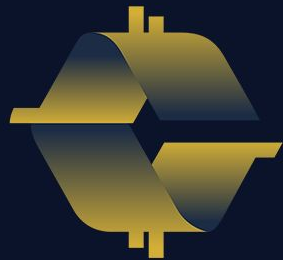
3종 장애 특성

REVERT · TIMEOUT · REORG

발생 시점과 재시도 가능 여부

§ 1

장애 분류



COINCRAFT
ACADEMY

3종 TX 장애 비교

§ 1 · 장애 유형별 발생 시점 · 원인 · 대응 전략

유형	발생 시점	원인	재시도?	대응
REVERT	블록 포함 후	컨트랙트 조건 미충족	❌ 금지	즉시 FAILED + reason 저장
TIMEOUT	mempool 대기 중	가스비 부족	✅ gas bump	PENDING 유지 → gas bump 후 재전송
REORG	블록 포함 후	체인 재편성	✅ 대기 후	REORGED → 5블록 대기 후 재확인

핵심 구분: 같은 FAILED처럼 보여도 원인이 다르면 대응이 달라진다 — REVERT는 원인 수정 없이 재시도 불가, TIMEOUT·REORG는 재시도 가능

REVERT 발생 원인

§ 1 · 컨트랙트 require / revert 조건 미충족

대표 REVERT reason 패턴

잔액 부족

```
ERC1155: burn amount exceeds  
balance
```

권한 없음

```
AccessControl: account 0x... is  
missing role
```

일시정지

```
Pausable: paused
```

기타 비즈니스 조건

컨트랙트별 커스텀 require 메시지

REVERT의 두 가지 특성

- > 가스는 소비됨 — 실행은 시도됐다
- > 상태는 롤백 — 블록체인 상태 변경 없음
- > TX는 블록에 포함됨 — onchain에 기록

주의: REVERT는 "요청이 틀렸다"는 뜻. 같은 조건으로 재시도해도 또 REVERT — 반드시 원인을 수정해야 한다

VASP 위탁에서도 REVERT 처리가 필요한가?

§ 1 · "VASP가 TX를 보내주는데 우리가 왜 REVERT를 처리해야 하는가"

① 결과 해석은 우리 책임

- VASP = TX 제출 + Webhook 전달까지
- { status: 'failed', revertReason } 수신 후
- DB FAILED 전이 · reason 저장 · 운영팀 알림
- → 우리 `handleTxRevert` 가 담당

② Webhook 누락 → 폴링 발동

- Webhook 지연·누락 시 `pollStaleRequests()`
- `IBlockchainAdapter.getReceipt()` 직접 호출
- `receipt.status = 'reverted'` 우리가 해석
- → VASP를 거치지 않는 경로

③ 재시도 정책은 비즈니스 판단

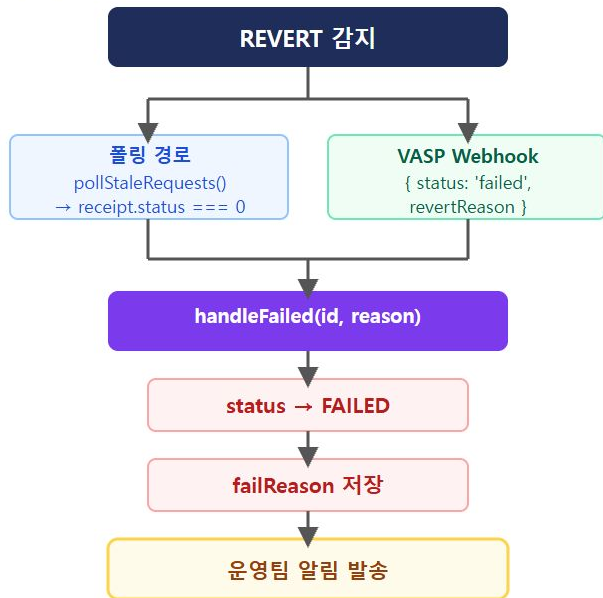
- VASP는 "실패했다"는 사실만 전달
- PAUSED-BALANCE-ACCESS 분류
- → 운영 조치 결정은 우리 도메인
- → `classifyRevertReason`

VASP 위탁 = TX 제출·모니터링 위임. 실패 결과를 받아 상태머신을 전이시키고 reason을 기록하고 운영 조치를 결정하는 것은 여전히 우리 코드의 책임이다.

REVERT 후 처리 정책

§ 1 · 즉시 FAILED 전이 + 자동 재시도 금지

처리 순서



재시도 정책

- 자동 재시도 금지 — 원인 수정 없이 재시도 = 가스비 낭비
- 운영 플로우 — 알림 → 원인 파악 → 수동 재발행

reason 분류별 해결 방향

분류	해결 방향
BALANCE	잔액 보충 후 재발행
PAUSED	컨트랙트 일시정지 해제
ACCESS	권한 부여 후 재발행

REVERT reason을 저장해야 운영팀이 원인을 파악할 수 있다 — reason 없이 FAILED만 기록하면 디버깅 불가

02

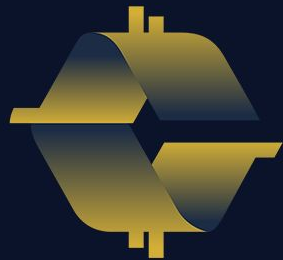
REVERT 처리 구현

handleFailed · reason 분류

InvalidStatusTransitionError 방어

§ 2

구현



COINCRAFT
ACADEMY

handleFailed 흐름

§ 2 · TxStateMachineService.handleFailed(requestId, failReason)

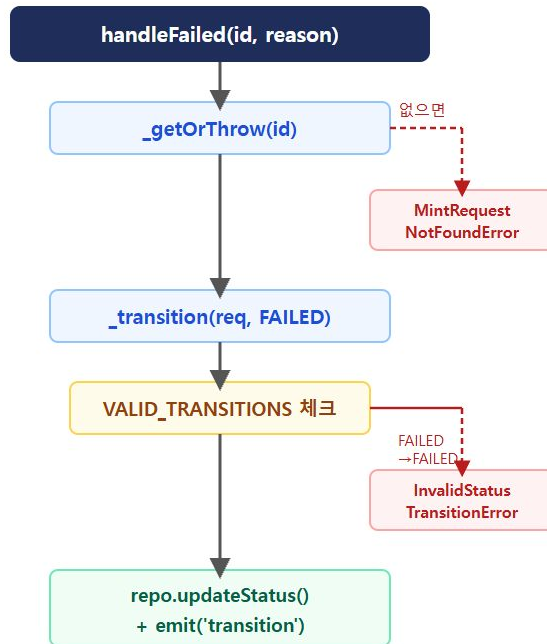
호출 시그니처

```
async handleFailed(  
  requestId: string,  
  failReason: string  
): Promise<void>
```

핸들러 특성

- › **가드 없음** — 어떤 상태에서든 FAILED 전이 시도
- › **VALID_TRANSITIONS 가드** — 이미 FAILED인 경우 차단
- › **종단 상태** — FAILED → 이후 전이 불가

실행 흐름



handleTxRevert 구현

§ 2 · VaspRecoveryService.handleTxRevert(requestId, txHash, reason)

실제 구현 코드

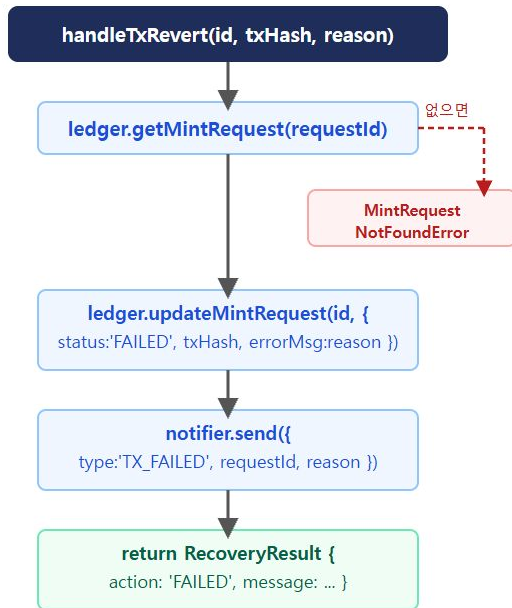
```
// VaspRecoveryService.ts:69
async handleTxRevert(
  requestId: string, txHash: string,
  reason: string,
): Promise<RecoveryResult> {
  const req = await this.ledger
    .getMintRequest(requestId);
  if (!req) throw new Error(`not found`);

  await this.ledger.updateMintRequest(
    requestId,
    { status: 'FAILED', txHash,
      errorMsg: reason });

  await this.notifier.send({
    type: 'TX_FAILED',
    requestId, txHash, reason });

  return { requestId,
    action: 'FAILED',
    message: `TX reverted: ${reason}` };
}
```

단계별 흐름



handleFailed와 차이: 상태머신 전이만 vs ledger + notifier 동시 처리

REVERT reason 분류

§ 2 · classifyRevertReason(reason: string) 패턴

분류 함수 구현

```
function classifyRevertReason(  
  reason: string  
) : 'BALANCE' | 'PAUSED' | 'ACCESS' | 'UNKNOWN' {  
  
  if (reason.includes('insufficient balance'))  
    return 'BALANCE';  
  if (reason.includes('Pausable: paused'))  
    return 'PAUSED';  
  if (reason.includes('AccessControl'))  
    return 'ACCESS';  
  
  return 'UNKNOWN';  
}
```

분류별 운영 대응

분류	reason 패턴	대응
BALANCE	insufficient balance	잔액 보충 후 재발행
PAUSED	Pausable: paused	컨트랙트 재개
ACCESS	AccessControl	권한 부여
UNKNOWN	기타	수동 조사 필요

reason 분류 없이는 모든 REVERT를 동일하게 처리하게 됨 — 분류가 운영 자동화의 핵심

운영 코드 — raw reason 그대로 보존

§ 2 · classifyRevertReason은 실습 전용 — 운영에는 없다

VaspRecoveryService.ts (실제 운영)

```
async handleTxRevert(  
  requestId: string,  
  txHash: string,  
  reason: string // ← 원문 그대로  
): Promise<RecoveryResult> {  
  const req = await this.ledger  
    .getMintRequest(requestId);  
  
  await this.ledger.updateMintRequest(requestId, {  
    status: 'FAILED',  
    txHash,  
    errorMsg: reason, // 분류 없이 저장  
  });  
  
  await this.notifier.send({  
    type: 'TX_FAILED',  
    requestId, txHash,  
    reason, // 원문 전달  
  });  
}
```

처리 흐름

1. DB 저장

errorMsg = reason — 원문 문자열 그대로 보존

2. 알림 발송

notifier.send({ type: 'TX_FAILED', reason })

→ 운영팀 Slack·대시보드에 원문 노출

3. 운영팀이 직접 판단

"잔액 부족이네 → 보충 후 재발행"

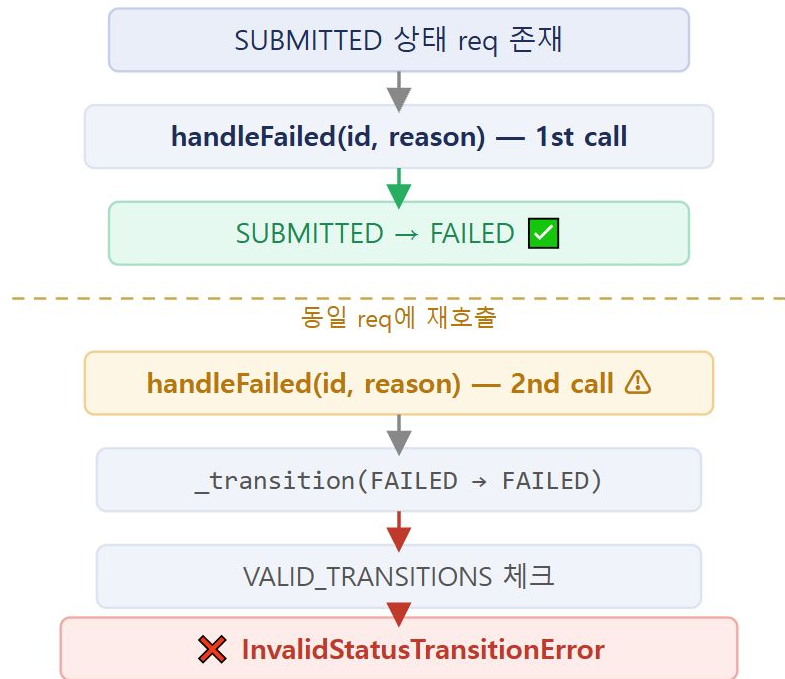
"AccessControl → 권한 확인"

classifyRevertReason은 실습 전용

Phase 1 = raw reason 보존 / Phase 2 = 분류 자동화 (미구현)

InvalidStatusTransitionError

§ 2 · FAILED는 종단 상태 — 재전이 불가



에러 메시지 및 의미

```
InvalidStatusTransitionError:  
"Invalid status transition:  
  FAILED → FAILED"
```

왜 명시적 에러인가?

- > FAILED는 종단 상태 — 한 번 실패하면 상태 변경 불가
- > 조용한 무시는 위험 — 이중 호출이 정상처럼 보임
- > 에러 = 버그 신호 — 로직 오류를 명확히 드러냄
- > 실습 [4] 확인 포인트 — expect(fn).rejects.toThrow()

이중 FAILED 전이 시도 → 조용히 무시가 아닌 명시적 에러 — 버그 탐지 가능

03

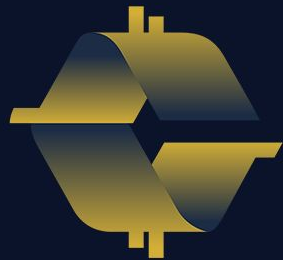
실습

npm run exercise:s18:answer

3종 장애 · handleTxRevert · reason 분류 · 상태 방어

§ 3

실습



COINCRAFT
ACADEMY

실습 [1] 3종 장애 시뮬레이션

§ 3 · REVERT · TIMEOUT · REORG 독립 처리 확인

REVERT 시뮬레이션

- receipt.status === 0 감지
- handleFailed 호출
- status → **FAILED**
- failReason에 'revert' 포함

TIMEOUT 시뮬레이션

- mempool 대기 시간 초과
- status → **PENDING** 유지
- txHash 변경 (gas bump)
- 새 nonce로 재전송

REORG 시뮬레이션

- MINED → **REORGED** 전이
- 5블록 대기
- 재확인 후 MINED or FAILED
- confirmations 재계산

검증 포인트: 각 장애 유형이 독립적으로 처리됨 — REVERT가 TIMEOUT 로직을 실행하거나, REORG가 즉시 FAILED가 되지 않아야 한다

실습 [2] handleTxRevert

§ 3 · SUBMITTED 상태 req → handleFailed 호출 → FAILED 전이 확인

테스트 시나리오

```
// 1. SUBMITTED 상태 req 준비
const req = await service.createRequest(params);
await service.handleSubmitted(req.id, txHash);

// 2. REVERT 감지 → handleFailed 호출
await service.handleFailed(
  req.id,
  'revert: Pausable: paused'
);

// 3. 결과 확인
const updated = await repo.findById(req.id);
expect(updated.status).toBe('FAILED');
expect(updated.failReason).toBe(
  'revert: Pausable: paused'
);
```

기대 결과

- > status → FAILED
- > failReason → 'revert: Pausable: paused'
- > observer → FAILED 구독자에게 이벤트 전달

observer 이벤트 확인

```
const events: TransitionEvent[] = [];
service.on('transition', e => events.push(e));

// handleFailed 호출 후
expect(events[0].to).toBe('FAILED');
expect(events[0].meta.failReason)
  .toContain('Pausable');
```


실습 [3] reason 분류

§ 3 · classifyRevertReason — 3가지 패턴 + UNKNOWN

테스트 케이스

```
// BALANCE
expect(classifyRevertReason(
  'ERC1155: insufficient balance'
)).toBe('BALANCE');

// PAUSED
expect(classifyRevertReason(
  'Pausable: paused'
)).toBe('PAUSED');

// ACCESS
expect(classifyRevertReason(
  'AccessControl: account 0x123 is missing role'
)).toBe('ACCESS');

// UNKNOWN
expect(classifyRevertReason(
  'unknown error'
)).toBe('UNKNOWN');
```

분류 결과 매핑

입력 reason	분류
insufficient balance 포함	BALANCE
Pausable: paused 포함	PAUSED
AccessControl 포함	ACCESS
위 패턴 없음	UNKNOWN

순서 중요: includes 체크는 위에서 아래로 — 더 구체적인 패턴을 먼저 확인해야 오분류 방지

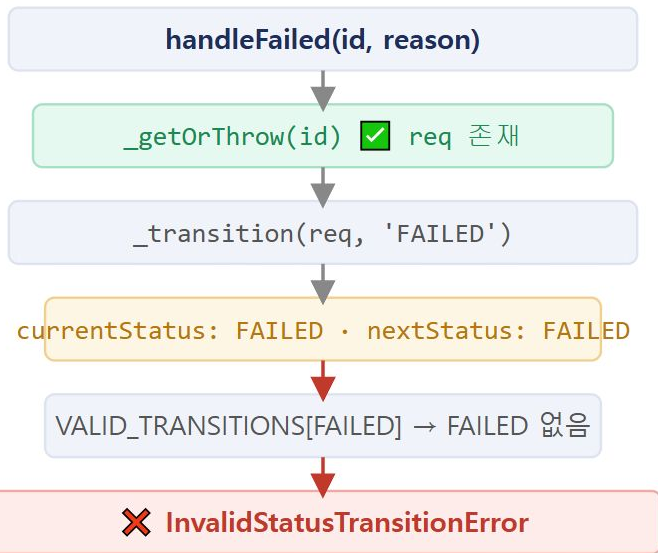
실습 [4] 잘못된 상태 전이

§ 3 · FAILED 상태에서 handleFailed 재호출 → 에러 발생 확인

테스트 시나리오

```
// 1. 정상 FAILED 전이
await service.handleFailed(
  req.id, 'revert: paused'
);
// status === 'FAILED'

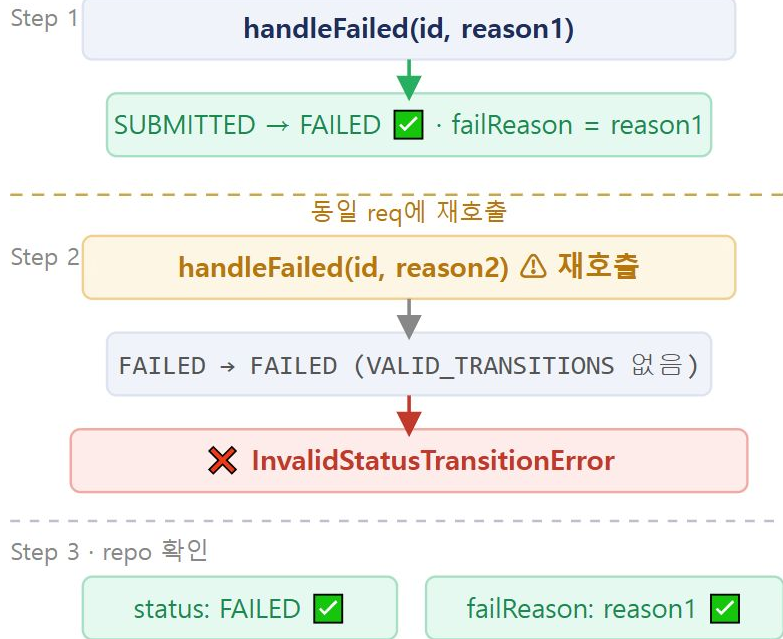
// 2. FAILED 상태에서 재호출
await expect(
  service.handleFailed(
    req.id, 'another reason'
  )
).rejects.toThrow(
  InvalidStatusTransitionError
);
```



FAILED는 종단 상태 — 재전이 불가. 이 에러가 발생하면 호출 로직에 버그가 있다는 신호

실습 [5] FAILED 중복 호출 안전성

§ 3 · 이중 FAILED 전이 → 명시적 에러로 처리



설계 의도

- › 조용히 무시하지 않는다 — 에러로 명시적 실패
- › 원본 reason 보존 — 최초 REVERT 원인이 덮이지 않음
- › 이중 호출 = 로직 버그 — 에러로 즉시 탐지

FAILED 재기록 시도는 명시적 에러 — 운영 중 동일 TX에 여러 이벤트가 도달하는 경합 조건 탐지에 유리

비교: 멱등성(idempotent) 처리로 조용히 무시할 수도 있지만, 명시적 에러가 디버깅에 유리 — 설계 선택의 문제

핵심 정리

S18 · TX REVERT 원인과 안전한 복구 설계

장애 유형	즉각 대응	재시도	최종 상태
REVERT	즉시 FAILED + reason 저장	❌ 금지	FAILED
TIMEOUT	gas bump 후 재전송	✅ 가능	PENDING → MINED
REORG	5블록 대기 후 재확인	✅ 가능	MINED or FAILED

REVERT 핵심 원칙: "REVERT는 원인을 고쳐야 한다. reason 저장이 전부다." — reason 없이는 운영팀이 조치 불가

상태 방어 원칙: FAILED는 종단 상태. 이중 전이 시도는 조용히 무시 대신 `InvalidStatusTransitionError`로 명시적 실패

다음 세션 S19 예고: MINED 후 CONFIRMED 전이 — confirmation count 추적 · 재편성(REORG) 후 재확인 로직 설계